



THE SCHOOL OF AI



NAVTC

Python for Applied AI

Day 1 — Why this language, why now, and what it gives you that the tools alone cannot.
A parallel track for AI for Everyone trainees who want to go one level deeper.

Parallel track · 4 hours a day · No prior programming experience · 08 September 2026 · Muhammad Rizwan

BEFORE WE START

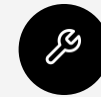
How this fits with AI for Everyone

Same trainees, same tools, one level down. You are not leaving that course — you are adding the layer underneath it.



AI for Everyone teaches

- Using the tools well.
- Writing prompts that produce sellable work.
- Building automations in n8n, no code required.
- Choosing the right tool for a job.



This track adds

- Reading and checking code an AI wrote.
- The step your no-code workflow cannot do.
- Working with data too messy for a spreadsheet.
- Calling AI models directly, from your own script.

Everything here connects back: Week 9 automations, Week 11 APIs, your capstone.

Four hours, and no installing anything

By the end of today you will have run your own code, broken it on purpose, and read the error it gave you.

1 Hour 1 · Why Python

The honest case in 2026, including the obvious objection: the AI can already write it.

2 Hour 2 · Where it sits

What applied AI actually runs on, where the no-code ceiling is, and what you will build here.

3 Hour 3 · Setup

Nothing to install. A browser, a Google account, and your first notebook.

4 Hour 4 · First contact

Run code, break it, read the error, fix it. That loop is the whole job.

The AI can write the code. It cannot be responsible for it.

That sentence is the entire argument for this track — and it is the same sentence you learned in the ethics session of the main course, applied one level down.

Three numbers worth knowing

Two of them say Python is winning. The third says something uncomfortable about how people are learning it.

26%

of the TIOBE language index in 2026 — the highest share ever recorded by any language

27%

of new code reported as AI-authored by early 2026, up from 22% six months before

50/67

quiz scores of developers who used AI versus those who hand-coded the same task

Sources: TIOBE index 2026 reporting · industry reporting on AI-authored code share, early 2026 · Anthropic controlled study of 52 developers, published January 2026: the AI-assisted group averaged 50% on a no-AI concept quiz, the hand-coding group 67%.

Five reasons, none of them nostalgia

Not "because programmers should know it". Each of these is something you will be paid for inside a year.

- | | | |
|---|------------------------------------|---|
| 1 | You can check the work | Python reads almost like English. That is why it is the language humans and AI co-write most easily — you can audit an AI's code in minutes. |
| 2 | It is the glue | Everything in applied AI has a Python interface. Models, data, files, APIs, cloud services. It is the control plane, even when it is not the engine. |
| 3 | It goes past the ceiling | Every no-code tool has a wall. n8n has a Code node; spreadsheets have a size limit; automations have a step they cannot express. Python is what is on the other side. |
| 4 | Data work lives here | Cleaning, joining, checking and summarising real business data. The unglamorous job that almost every client actually needs. |
| 5 | It is the door to the roles | AI application engineer, automation engineer, data roles. Every one of those job adverts assumes Python without saying so. |

When no-code is enough, and when it is not

THE JOB	STAY IN N8N OR A TOOL	REACH FOR PYTHON
Form to sheet to email	Yes — this is exactly what it is for	No need
Clean 12,000 messy rows	Painful and slow	Twenty lines, runs in seconds
One rule with six exceptions	A maze of IF nodes	A short function you can read
Call a model and reshape the reply	Works for simple cases	Needed once the output is structured
Something to hand a client to run	Tied to your account	A script or small app they own

Most real work is both. No-code for the plumbing, Python for the hard step.

Four places Python does the work

You will not be training models in this course. You will be doing the four things that actually pay.



Data

Read a messy file, clean it, join it, check it, summarise it. The most common paid task of all.



Calling models

Send text to an AI model from your own script and handle what comes back, reliably.



Automation

Scheduled jobs, file handling, moving things between systems that have no ready-made connector.



Small apps

A page or a tool a client can actually use, rather than a workflow only you can run.

What you will have built by the end

STAGE	WHAT YOU BUILD	WHY IT MATTERS
Foundations	Small scripts that read files and print clean output	You stop being afraid of the black screen
Data	A script that cleans a real messy spreadsheet	This alone is a sellable service
APIs	Your own script that calls an AI model and handles the reply	The step behind every AI product you have used
Automation	A scheduled job, and a Python step inside an n8n workflow	Where this track meets the main course
Delivery	One small tool a client could run themselves	A portfolio piece, not an exercise

HOURL 2 · THE RULE FOR THIS CLASSROOM

**Type it yourself first.
Ask the assistant second.**

You will use AI constantly in this work, and you will use it badly if you skip the part where you learned what the code does. This is the one classroom rule that is not negotiable.

Three passes, every single time

This is how a professional uses AI on code. It is also, by accident, the fastest way to learn.

1

Write it badly yourself

Attempt it before you ask anything. A wrong attempt tells you what you do not understand, which is the only useful information at this stage.

2

Ask for an explanation, not code

"Why does my loop print the last item five times?" teaches you. "Write me a loop" does not.

3

Retype what you accept

Never paste. Type the corrected version by hand. Your fingers learn what your eyes skip.

By Week 4 you may paste freely. You will have earned the ability to check it.

You are not installing anything today

Everything in the first weeks runs in a browser tab. This removes the single most common reason beginners quit.



Google Colab

Python in a browser.
Nothing to install, nothing to
configure, works on a weak
laptop.



A Google account

The one you already made
in Week 1. Your notebooks
save to your own Drive.



Survives outages

The work lives online. A
power cut costs you the last
few minutes, not the
project.



Notebooks, not files

Code in small blocks you
run one at a time, with your
notes in between. Ideal for
learning.

What everything else is called

YOU WILL HEAR	WHAT IT IS	WHEN YOU NEED IT
Colab / Jupyter	Python notebooks in a browser	Now. Weeks one to four of this track
VS Code	A proper code editor on your own machine	Later, once you write real scripts
uv	The fast modern way to install packages and manage versions	When you move off Colab. It has largely replaced older tooling
Python 3.13 / 3.14	The stable versions everything supports today	Use these. Colab picks one for you
Python 3.15	Final release due 1 October 2026	Not yet. Let the libraries catch up first

Your first notebook

Open colab.research.google.com, sign in, and make a new notebook. Then type — do not paste — each of these.

TYPE THIS INTO THREE SEPARATE CELLS

```
name = "Bilal"  
city = "Lahore"  
print("Hello,", name + "!", "Welcome from", city + "!")
```

Change the names and the numbers to your own. Run each cell as you go.

Your first notebook

Open colab.research.google.com, sign in, and make a new notebook. Then type — do not paste — each of these.

TYPE THIS INTO THREE SEPARATE CELLS

```
orders = [120, 340, 90, 560]

print("Order Summary")
print("-----")
print("Total Orders:", len(orders))
print("Total Amount:", sum(orders))
print("Biggest Order:", max(orders))
```

Change the names and the numbers to your own. Run each cell as you go.

Now break it on purpose

Delete a quotation mark and run it. Read what Python says. Errors are instructions, not punishment.

- | | | |
|---|---------------------------------|--|
| 1 | Read the last line first | The bottom line of an error names the problem. Everything above it is the path Python took to get there. |
| 2 | Find the line number | Python tells you where it stopped — which is usually one line after where you actually made the mistake. |
| 3 | Say it in plain words | "It expected a closing quote and did not find one." If you can say it, you can fix it. |
| 4 | Then ask, if you need to | Paste the error, not just the code, and ask why it happened. That is pass two from the rule earlier. |

Something small and actually useful

Six lines that do a job you have done by hand: turning a messy list into a clean one.

TYPE IT, RUN IT, THEN CHANGE THE LIST TO REAL NAMES FROM YOUR OWN WORK

```
names = ["  ali khan ", "SARA AHMED", "bilal  ", " Ayesha Khan"]

for raw in names:
    clean = raw.strip().title()
    print(clean)
```

You just did in six lines what takes ten minutes in a spreadsheet.

Scale that to four thousand rows and it is the same six lines. That is the whole argument for this track.

Twenty minutes, not two hours

Daily practice beats weekend marathons in programming more than in almost anything else.

1 Re-type both labs

From scratch, in a new notebook, without looking at the slides. Getting stuck is the point — note where.

2 Break three things

Cause three different errors on purpose and write down, in your own words, what each one told you.

3 Bring a messy list

Real names, products or places from your own work or family business. Next session we clean it properly.

Rule stands: type it yourself first, ask the assistant second.

END OF DAY 1

**You are not learning to type code.
You are learning to answer for it.**

Next · Day 2 — variables, lists and loops properly, reading a real file, and the first script that saves you an hour of work you currently do by hand.